

Draftly Retrieval Engine — Methodology and Codebase Guide

A walk-through of how the retrieval engine is built: where the data comes from, how it is turned into a searchable store, how a question travels through the code to a cited answer, and where every piece lives in the repository. Written for someone starting on the project who wants the whole shape before touching the code.

- **Scope of this document:** the statute-and-amendment retrieval engine that ships today, plus the surrounding data pipeline and case-law work it depends on.
 - **Source of truth:** the code under `src/draftly/retrieval/`, the data pipeline in `notebooks/`, and the design record in `retrieval-engine-plan.md`. Where the plan describes something not yet built, this guide says so.
-

1. What the engine is, in one paragraph

Draftly answers Sri Lankan conveyancing questions against a **closed, curated legal corpus** — 57 statutes and 18 amendments in the current demo. The design choice that drives everything: **deterministic structure does the legal work, and search only does recall**. Curated topic tables scope the search, a statute cross-reference graph pulls in structurally-linked sections, BM25 (a classic keyword ranking method) handles lexical matching, an optional embedding channel catches paraphrase, and a bounded language model writes the final answer using **only** the retrieved text. Every claim in an answer must carry a citation that resolves against the corpus; if nothing relevant is found, the engine abstains rather than guessing. The reasoning behind each of these choices, weighed against twelve research papers, is recorded in `retrieval-engine-plan.md`.

2. Why this design (and not a vector database)

Most legal retrieval systems reach for embeddings and a vector database. This project deliberately does not lead with that. The plan document lays out the argument in full; the short version:

- **Similarity is not legal correctness.** Whether a section is *in force on a given date* cannot be expressed as "how similar is this text." Two amended versions of the same section are near-identical in embedding space, so similarity cannot pick the one that actually applies.
- **We already own what other systems approximate.** The topic-to-statute tables are curated by hand. The case-to-section links are extracted deterministically and quote-backed. The statute registry is closed and complete. Papers spend heavy machinery guessing these; here they are ground truth.
- **Deterministic systems fail loudly; probabilistic ones fail fluently.** When a keyword lookup finds nothing, it returns nothing. A vector search always returns its closest guess, even when the guess is

wrong — the dangerous failure mode for legal work.

Embeddings still exist in the codebase, but as a **fallback recall channel** fused alongside BM25, never as the primary path. If the key is absent or the network is down, search silently degrades to keyword-only.

3. Repository map

The parts that matter for retrieval, top to bottom:

```

draftly/
├── src/draftly/retrieval/           # THE ENGINE – the Python package
│   ├── paths.py                   # where every data file lives
│   ├── corpus.py                  # load statutes + topics from CSV
│   ├── section_parser.py          # split a statute document into sections
│   ├── index.py                   # build the SQLite + full-text search index
│   ├── search.py                  # the multi-channel retrieval + fusion
│   ├── graph.py                   # statute cross-reference graph + PageRank
│   ├── embeddings.py              # optional dense (semantic) channel
│   ├── question_analysis.py       # split a question into sub-queries, hints
│   ├── answering.py               # bounded LLM answer + citation checking
│   ├── evaluation.py              # dev retrieval metrics
│   ├── qa_evaluation.py           # run the exam-question harness
│   └── __main__.py                # the CLI (build / search / ask / evaluate)
│
├── data/
│   ├── legal-sources/             # RAW + CURATED inputs
│   │   ├── library/               # collected PDFs (statutes, case-law...)
│   │   ├── library-markdown/      # converted to text
│   │   ├── topics/                # 20 curriculum topic folders
│   │   └── manifests/             # the tracked CSV registries (see §4)
│   ├── processed/                 # BUILT store (gitignored, rebuildable)
│   │   ├── documents.csv          # one row per source document
│   │   ├── source-registry.csv    # statute metadata
│   │   ├── topics.csv             # 20 topics
│   │   ├── topic-sources.csv      # 174 topic-statute edges
│   │   ├── retrieval_eval_gold.csv (tracked – the dev gold set)
│   │   └── retrieval-indexes/     # the built SQLite index lives here
│   └── raw/                       # private demo case bundles (not published)
│
├── notebooks/
│   ├── data_processing_pipeline.ipynb # raw docs → processed store
│   ├── 01_bm25_retrieval_baseline.ipynb # retrieval baseline
│   └── 02_caselaw_extraction_validation.ipynb
│
├── scripts/                       # harvesting + case-law extraction tooling
│   └── case-law-information-extraction/ # rule extraction (P4 of the plan)
│
├── apps/statute-retrieval/app.py # the Streamlit demo UI
├── evaluation/                    # gold labels, rubrics, saved run outputs
└── retrieval-engine-plan.md       # the design record and its evidence

```

A practical note on what is checked into git and what is not: the **manifests** under `data/legal-sources/manifests/` are tracked — they are the curated registries. The **built store** under `data/processed/` is gitignored because it is rebuildable from the PDFs, and the harvested case-law text is gitignored because it is large and, in places, copyrighted. So a fresh clone has the recipe and the registries, and you rebuild the store locally.

4. The data layer — how sources become a corpus

Retrieval is only as good as the store underneath it. The data moves through four stages, from a pile of PDFs to a set of clean CSVs the engine reads.

4.1 Collection — `data/legal-sources/`

Inputs are gathered by hand and by harvesting scripts:

- **Statutes and amendments** as PDFs under `library/statutes/` and `library/amendments/`.
- **Case law** harvested from public sources — CommonLII, the Internet Archive, and court sites — by the `scripts/harvest*_caselaw.py` scripts. The roadmap records the scale: 3,703 conveyancing judgments, a ~9,500-case index spanning 1872–2010, and 33 public-domain volumes.
- **Curriculum topics** — the 20 conveyancing topics as folders under `topics/`, each mapping to the statutes that govern it.

4.2 The manifests — the curated registries

These CSVs under `manifests/` are the human-owned backbone. They are tracked in git and everything downstream keys off them:

Manifest	Rows	What it holds
<code>source-registry.csv</code>	90	Every source document: title, act number, year, topics, URL, checksum
<code>topics.csv</code>	20	The curriculum topics
<code>topic-sources.csv</code>	174	Which statutes belong to which topic (the scoping table)
<code>conversion-registry.csv</code>	66	PDF → Markdown conversion status and quality notes
<code>case-law-commonlii-conveyancing.csv</code>	3,703	The reported conveyancing judgments
<code>case-law-citations.csv</code>	76	Case → statute-section citation links (verified subset)
<code>slr-modern-cases.csv</code>	630	Modern Sri Lanka Law Reports cases

The topic tables are the important curated asset: they let the engine **scope a search to the handful of statutes that actually govern a topic** before it does any keyword matching, which is where most of the

retrieval quality comes from.

4.3 Conversion — PDF to text

`scripts/convert_legal_sources_to_markdown.py` and the notebook turn each source PDF into normalized Markdown under `library-markdown/`, recording converter, version, and quality notes in `conversion-registry.csv`. OCR quality varies — old print and mixed Sinhala/English are the hard cases — so the registry flags documents that need review rather than trusting every extraction.

4.4 The processing pipeline — `notebooks/data_processing_pipeline.ipynb`

This notebook is the bridge from collected documents to the store the engine reads. Its stages:

- **Stage 0 — Discovery index:** enumerate every source document.
- **Stage 1 — Select text and audit quality:** pick the best text per source, flag weak OCR.
- **Stage 2 — Extract section mentions:** find statute and section references in the text.
- **Stage 3 — Attach topics:** join each document to its curated topics.
- **Stage 4 — Write retrieval records:** emit `documents.csv` and the pipeline summary.
- **Stage 5 — Evaluation set:** produce the reviewer-owned gold labels.
- **Stage 6 — Canonical normalized store:** the final `data/processed/` CSVs.

The output of this notebook is exactly what `src/draftly/retrieval/paths.py` points at: `documents.csv`, `source-registry.csv`, `topics.csv`, `topic-sources.csv`, and the gold set.

5. The engine internals — module by module

Now the code. Each module has one job; here is the chain in the order the data flows through it.

5.1 `paths.py` — the map

A short module that names every file the engine reads and where the built index lives. If a path changes, it changes here and nowhere else.

5.2 `corpus.py` — loading and validating inputs

`load_statute_documents()` reads `documents.csv`, keeps only rows with `status == "converted"` and a kind of statute or amendment, and then **asserts the count is exactly 57 statutes and 18 amendments**. If the number is off, or if any case-law document leaks into the statute set, it raises. This is a deliberate guard: the statutes-only demo must never quietly answer from case-law text.

`corpus_fingerprint()` hashes the input version tag, the three metadata CSVs, and every source file's size and modification time into a single SHA-256 digest. That fingerprint is how the engine knows whether its index is stale — change any input and the fingerprint changes, which triggers a rebuild.

`build_section_nodes()` ties it together: for each document it reads the text, resolves the topics, and calls the section parser to split it into sections.

5.3 `section_parser.py` — one document into many sections

A statute is one file, but retrieval works best over **sections**, not whole acts. This module is a careful regex-based parser that finds section boundaries (`2. Interpretation` , heading-then-number patterns, bare `2.` lines) and rejects false boundaries — page numbers, law-report citations, sub-clause markers.

Three details worth knowing:

- **Document fallback:** if a document yields fewer than two clean sections (usually bad OCR), the whole document becomes a single node flagged `document_fallback` , so it is still searchable but visibly lower-confidence.
- **Aliases:** an amendment that says "amendment of section 28" gets an *alias node* under the principal act's section number, so a search for that section surfaces the amending text. The alias's ID is a retrieval convenience, and the hit carries a note saying so.
- **Quality quarantine:** known-bad extractions (for example, the intestate succession shares table whose PDF uses interleaved columns) are flagged with a warning instead of being served as clean evidence.

5.4 `index.py` — building the searchable index

The parsed section nodes are written into a **SQLite database** with two tables: a `sections` table holding the full metadata and a `sections_fts` virtual table using SQLite's **FTS5 full-text search** for fast keyword lookup.

The build is designed to be safe and cheap to repeat:

- It is keyed by the corpus fingerprint. If an index for the current fingerprint already exists, `build_index()` reuses it and returns immediately.
- The build writes to a temporary file and atomically swaps it in, behind a file lock, so concurrent processes or a crashed build cannot corrupt the active index. An `active.json` pointer records which index file is live.

5.5 `question_analysis.py` — understanding the question

Before searching, a question is analyzed. An exam question often has several parts and several distinct legal issues; treating it as one bag of words retrieves poorly. This module splits a question into **sub-queries**, detects which statutes are likely relevant (**source hints**, from a keyword table), and flags **known corpus gaps** — topics the statutes-only demo genuinely cannot answer, so the engine can abstain honestly instead of retrieving something loosely related.

5.6 `search.py` — the heart: multi-channel retrieval and fusion

This is where a query becomes a ranked list of sections. It runs **several retrieval channels** and merges them. For each sub-query it collects:

1. **Direct lookup** — if the query names a section explicitly (s.2 Prevention of Frauds , a SRC001:s2 ID, or a section range), fetch it by ID. Highest confidence, so it is scored to sort first.
2. **Lexical (BM25) lookup** — the FTS5 keyword search, with a query builder that drops stopwords, expands known phrases ("deed valid" also searches "notary", "witnesses", "executed"), and reranks by soft topic match.
3. **Dense lookup** — the optional embedding channel (see 5.8), fused in only when embeddings are available.
4. **Source-hinted lookup** — for each statute the analysis flagged, a scoped search inside just that statute, with curated hint terms.
5. **Seeded entry points** — for hinted statutes, hand-picked "start here" sections.

All of these lists are combined with **Reciprocal Rank Fusion (RRF)** — a standard method that scores each result by its *rank* in each list rather than by raw scores, which lets very different channels (keyword, semantic, direct) be merged fairly. Curated seeds and source-hint matches get a small weight bonus so the human-curated signal wins ties. The fused list is de-duplicated by passage so near-identical OCR variants do not crowd the results.

Finally, the fused top results are handed to the graph for expansion, and the whole thing is fused once more.

5.7 graph.py — the statute cross-reference graph

Statutes are graphs: a section says "subject to section 5," an amendment targets a section of its parent act, and an interpretation section defines the terms every sibling uses. This module **builds that graph deterministically from the section text** — no language model — with three edge types:

Edge	Weight	Meaning
cross_ref	1.0	Section body cites "section N" of the same act
amendment	1.2	Amendment alias node ↔ the principal section it changes
definition	0.25	Section → its act's interpretation/definitions section

Then `expand_seeds()` runs a small **Personalized PageRank** walk from the retrieval hits. PageRank is the "importance by connection" algorithm; the *personalized* variant biases it toward the seed sections. The effect: a section that no query word matched, but that the top hits structurally depend on — the provision they are "subject to", the amendment that rewrites them, the definitions their terms rely on — gets surfaced. The graph is cached per fingerprint.

5.8 embeddings.py — the optional semantic channel

Keyword search misses paraphrase ("sign abroad" vs "outside Sri Lanka", "reserve fund" vs "sinking fund"). This module embeds each section once with Gemini embeddings, caches the vectors as float32 blobs in a SQLite file keyed by the corpus fingerprint, and returns the nearest sections for a query. It is built to **degrade gracefully**: no API key, no network, or a failed build all yield empty results, and

`search.py` simply continues keyword-only. It can also be disabled with the `DRAFTLY_DISABLE_DENSE` environment variable.

5.9 `answering.py` — from evidence to a cited, checked answer

The `ask` path retrieves evidence, then asks a **bounded** language model (Gemini by default) to write an answer using **only the supplied sections and facts stated in the question**. The discipline is enforced in code, not trusted to the model:

- The model must return structured JSON — an outcome (`answered` / `partial` / `abstained`), a list of claims, and each claim's citations.
- **Every citation is checked** against the retrieved section IDs. A claim citing something that was not in the evidence is dropped.
- If retrieval found nothing, if the question is outside the supported domain, or if it hits a known corpus gap, the engine **abstains** before ever calling the model.
- If the API key is missing or the call fails, it returns an **evidence-only** answer — the retrieved sections with no generated prose — rather than inventing text.

Each answer ships a machine-readable record: the outcome, the claims and their citations, the retrieval queries used, the corpus fingerprint, and a trace ID.

6. The life of a question, end to end

Putting the modules together, here is what happens when you run `draftly ask "What makes a deed valid?"`:

```

question
|
▼
[question_analysis]  split into sub-queries, detect source hints + gaps
|
▼
[index.build_index]  reuse the SQLite index if the fingerprint matches,
|                    otherwise parse sections and rebuild it
|
▼
[search]  per sub-query, run and collect ranked lists:
|          • direct section lookup
|          • BM25 keyword search (FTS5)
|          • dense/embedding search   (if available)
|          • source-hinted scoped search
|          • curated seed sections
|          → fuse with Reciprocal Rank Fusion
|          → [graph] Personalized PageRank expansion over cross-references
|          → fuse again, de-duplicate
|
▼
ranked sections (evidence)
|
▼
[answering]  abstain if no evidence / out of domain / known gap
|            else → bounded LLM writes claims from evidence only
|            → drop any claim whose citation does not resolve
|
▼
cited answer + JSON provenance (outcome, claims, citations, trace, fingerprint)

```

The `search` command stops at "ranked sections" and prints them. The `ask` command runs the whole chain.

7. Case-law extraction (the parallel data track)

The statute engine is the shipping demo, but a second track builds the case-law knowledge the fuller engine will use. It lives in `scripts/case-law-information-extraction/` and extracts the **legal rule** (the ratio) from each judgment into a quote-grounded table. Its method mirrors the engine's philosophy — deterministic first, model second, validator on top:

- **Track A (reported cases):** pull the rule straight from the `Held:` block of the headnote. No model, no cost.
- **Track B (unreported / no headnote):** a bounded model extracts a rule over a windowed extract, and **must return a supporting quote that appears verbatim in the judgment** or the rule is

rejected.

- **The grounding gate:** `validate.py` throws out any rule whose quote is not a real substring of the judgment, and nulls any statute section that is not in the registry. The validator, not the model, guards correctness. Every rule is marked `unverified` until a lawyer signs off.

The outputs (`rules.csv` , `case_meta.csv`) are gitignored because they contain derived and copyrighted headnote text.

8. Evaluation — how "it works" is measured

Retrieval claims are checked, not asserted. The pieces:

- `evaluation.py` — development retrieval metrics against the ten unverified dev labels in `data/processed/retrieval_eval_gold.csv` .
- `qa_evaluation.py` — runs the full question harness. The `evaluate-questions` command runs all 18 exam questions and 73 sub-questions from `src/questions.md` . It measures retrieval robustness; it does **not** claim legal correctness, because that needs lawyer-verified gold labels.
- `evaluation/runs/` — saved outputs from each run (metrics, per-question scores, configs), so results are reproducible and comparable across changes.
- `notebooks/01_bm25_retrieval_baseline.ipynb` — the retrieval baseline the engine is measured against.

The honesty rule throughout: no legal-correctness number is reported without lawyer-verified gold. Everything currently labelled is "unverified" and says so.

9. Running it yourself

From the project root, with `uv` (the Python environment manager this project uses):

```
# one-time: install dependencies
uv sync

# build (or refresh) the statute index
uv run python -m draftly.retrieval build

# see retrieval only – ranked sections, no generated prose
uv run python -m draftly.retrieval search "What makes a deed valid?"

# full answer – needs GEMINI_API_KEY in a .env file
uv run python -m draftly.retrieval ask "What makes a deed valid?"

# the demo web UI
uv run streamlit run apps/statute-retrieval/app.py

# evaluation
uv run python -m draftly.retrieval evaluate
uv run python -m draftly.retrieval evaluate-questions
```

Without a `GEMINI_API_KEY`, `search` works fully and `ask` returns evidence-only answers. The embedding channel also stays off without the key, so retrieval runs keyword-plus-graph only — which is the intended default anyway.

10. Where to read next

- `retrieval-engine-plan.md` — the design record: every adopt/reject call weighed against the research, the planned data model, the query taxonomy, and the build order. Read this to understand *why*.
- `case-law-extraction-plan.md` — the case-law rule-extraction design.
- `roadmap.md` — the three team workstreams (OCR, corpus, templates) and how they fit.
- `README.md` — the shortest path to running the demo.
- **The code** — start at `src/draftly/retrieval/__main__.py`, then follow `search.py`, which calls almost everything else.

This guide describes the engine as built in `src/draftly/retrieval/`. The richer data model in the plan — the unified edge list, dated section versions, authority ranking — is designed and partly prototyped but not all wired into the shipping statutes-only demo. Where this guide and the plan differ, the plan is the intent and this guide is the current state.